

Pokémon TCG Legacy — Configuration Guide

A living reference for every JSON-driven knob in the game. Each section lists the **field name**, the **allowed values**, the **file(s) that consume it**, and a **minimal example**.

Keep this file open while editing data files in `NPC_and_Opponent_Data/` , `Card_Set_Data/` , and `Player_Data/` .

1. Battle Conditions (Deck Restrictions)

Opponents can demand the player meet specific deck requirements before a battle starts. The restriction is attached to an opponent entry inside any `NPC_and_Opponent_Data/<Map>_<Day>_<Time>.json` file under the `opponents` array.

Validator: `Scripts/Global_Scripts/Deck_Validation_Helper.gd` **Popup UI:** `Scripts/Global_Scripts/Deck_Validation_Popup.gd` **Trigger:** `MapManager._on_yes_pressed` runs validation just before the battle scene loads. If validation fails, a popup (or text-only message) is shown and the battle is aborted.

1.1 Display modes

Mode	Triggered by	What the player sees
banned	<code>banned_card_ids</code> / <code>banned_card_names</code> hit	Full-screen grid titled " Banned cards " showing every card on the NPC's banlist
invalid	Structural rule violated (type, set, supertype, HP, etc.)	Full-screen grid titled " Cards in your deck that aren't allowed " showing the offending cards from the player's deck
missing	Required-card quantity not met	Standard messagebox text only, e.g. " <code>You can't battle me yet:</code> <code>• Need at least 4× Alakazam (have 0)</code>"

If multiple categories fail at once, priority is **banned > invalid > missing**. Missing-requirement lines are appended to the popup body when an `invalid` -mode popup is shown so the player sees everything at once.

1.2 Restriction schema

```
"restrictions": {  
  
  // ——— Strict blacklists (mode = "banned") ———  
  "banned_card_ids": ["base1-88", "base1-89"], // exact card ids  
  "banned_card_names": ["Professor Oak", "Bill"], // by name, matches across all sets  
  
  // ——— Required-quantity rules (mode = "missing") ———  
  "required_card_ids": [{"id": "base1-1", "min": 4}],  
  "required_card_names": [{"name": "Alakazam", "min": 4},  
                           {"name": "Psychic Energy", "min": 10}],  
  "min_stage_2_pokemon": 4, // at least N Stage-2 Pokémon  
  
  // ——— Structural rules (mode = "invalid") ———  
  "allowed_pokemon_types": ["Fire"], // Pokémon type whitelist  
  "allowed_energy_types": ["Fire"], // Energy cards matched by NAME  
  "allowed_set_ids": ["base1", "base2"], // card id prefix whitelist  
  "allowed_pokemon_name_prefixes": ["Dark "], // Pokémon name must start with one  
  "banned_pokemon_name_prefixes": ["Dark "], // Pokémon name must NOT start with one  
  "no_trainers": true,  
  "no_special_energies": true,  
  "special_energies_only": true, // basic energies forbidden  
  "basic_pokemon_only": true, // Stage 1/2 forbidden  
  "max_hp": 60, // Pokémon HP cap  
  "delta_species_only": true, // Pokémon name must contain " δ"  
  "no_ex_cards": true // any subtype "ex" forbidden  
}
```

1.3 Field reference

Field	Type	Notes
<code>banned_card_ids</code>	<code>String[]</code>	Exact card ids (<code>"base1-88"</code>). Case-insensitive.
<code>banned_card_names</code>	<code>String[]</code>	Card names. Matches across all sets, so <code>"Professor Oak"</code> catches every reprint.
<code>required_card_ids</code>	<code>[{id, min}]</code>	Per-card minimum quantity; matches one exact id.
<code>required_card_names</code>	<code>[{name, min}]</code>	Per-name minimum quantity; sums copies across all sets. Use this for energies (e.g. <code>"Psychic Energy"</code>) because energy cards reprint into many sets.
<code>min_stage_2_pokemon</code>	<code>int</code>	Total Stage-2 Pokémon copies required.
<code>allowed_pokemon_types</code>	<code>String[]</code>	Pokémon <code>types[]</code> field must intersect. Values: <code>Fire</code> , <code>Water</code> , <code>Grass</code> , <code>Lightning</code> , <code>Psychic</code> , <code>Fighting</code> , <code>Colorless</code> , <code>Darkness</code> , <code>Metal</code> , <code>Dragon</code> , <code>Fairy</code> .
<code>allowed_energy_types</code>	<code>String[]</code>	Matched off the energy card's name (<code>"Fire Energy"</code> → <code>Fire</code>). Different sets share the same name, so cross-set Fire Energies all pass.
<code>allowed_set_ids</code>	<code>String[]</code>	Card id prefix (e.g. <code>base1</code> , <code>base5</code> , <code>gym1</code>). Applies to all card types.
<code>allowed_pokemon_name_prefixes</code>	<code>String[]</code>	Case-insensitive name prefix list. Use <code>"Dark "</code> (with trailing space) for Team-Rocket Dark Pokémon, <code>"Light "</code> for Neo Destiny Light Pokémon.
<code>banned_pokemon_name_prefixes</code>	<code>String[]</code>	Reverse of the above.
<code>no_trainers</code>	<code>bool</code>	Forbid all Trainer cards.
<code>no_special_energies</code>	<code>bool</code>	Forbid any energy with subtype <code>"Special"</code> .
<code>special_energies_only</code>	<code>bool</code>	Forbid any energy with subtype <code>"Basic"</code> .
<code>basic_pokemon_only</code>	<code>bool</code>	Only <code>"Basic"</code> or <code>"Baby"</code> subtype Pokémon.
<code>max_hp</code>	<code>int</code>	Pokémon with HP > N are invalid. Trainer/Energy cards ignore this.
<code>delta_species_only</code>	<code>bool</code>	Pokémon name must contain " <code>δ</code> " (the delta-species marker).
<code>no_ex_cards</code>	<code>bool</code>	Any card whose <code>subtypes</code> array contains <code>"ex"</code> is invalid.

1.4 Example: gym challenge

```
{
  "name": "Gym Leader Erika",
  "meet_text": "As part of the Gym Challenge you may only use Grass Pokémon. No Trainers either!",
  // ... other opponent fields ...
  "restrictions": {
    "allowed_pokemon_types": ["Grass"],
    "no_trainers": true
  }
}
```

1.5 Set id reference (for `allowed_set_ids`)

Set id	Set name
<code>base1</code>	Base Set
<code>base2</code>	Jungle
<code>base3</code>	Fossil
<code>base5</code>	Team Rocket
<code>basep</code>	Wizards Promo
<code>gym1</code>	Gym Heroes
<code>gym2</code>	Gym Challenge
<code>neo1 – neo4</code>	Neo Genesis / Discovery / Revelation / Destiny
<code>ecard1 – ecard3</code>	Expedition / Aquapolis / Skyridge
<code>ex1 – ex16</code>	EX era
<code>np</code>	Nintendo Promo
<code>pop1 – pop6</code>	POP series
<code>si1</code>	Southern Islands

2. NPC & Opponent Spawning Conditions

Any NPC or opponent entry may carry a `condition` block. The entry only spawns if the condition evaluates to **true**.

Evaluator: `MapManager._evaluate_condition` (MapManager.gd ≈ line 311) **Special case:** During the immediate return-to-map after a battle, a just-beaten opponent stays spawned long enough to play the result text, even if their `opponent_not_defeated` condition would normally hide them now.

2.1 Condition types

type	Extra fields	Returns true when
<code>opponent_defeated</code>	<code>target: String</code>	The named opponent has been beaten at least once
<code>opponent_not_defeated</code>	<code>target: String</code>	The named opponent has NOT been beaten
<code>all_opponents_defeated</code>	<code>targets: String[]</code>	Every name in the list has been beaten
<code>not_all_opponents_defeated</code>	<code>targets: String[]</code>	At least one name in the list has NOT been beaten
<code>any_opponent_defeated</code>	<code>targets: String[]</code>	At least one name in the list has been beaten
<code>npc_met</code>	<code>target: String</code>	The named NPC has been talked to at least once
<code>npc_not_met</code>	<code>target: String</code>	The named NPC has NOT been talked to
<code>flag_set</code>	<code>flag: String</code>	<code>GameState.progress[flag]</code> is truthy
<code>flag_not_set</code>	<code>flag: String</code>	<code>GameState.progress[flag]</code> is falsy or missing

2.2 Examples

```
// Spawn only after Misty has been beaten
"condition": { "type": "opponent_defeated", "target": "Gym Leader Misty" }

// Spawn until the player completes ALL six Pikachu Fans
"condition": {
  "type": "not_all_opponents_defeated",
  "targets": [
    "Pikachu Fan Marina", "Pikachu Fan Skye",
    "Pikachu Fan Cami",   "Pikachu Fan Juniper",
    "Pikachu Fan Raye",   "Pikachu Fan Leaf"
  ]
}

// Spawn after a custom progress flag has been set elsewhere
"condition": { "type": "flag_set", "flag": "got_starter_deck" }
```

3. Movement Patterns

`pattern` controls how a spawned NPC or opponent moves around the map. The default is `"idle_random"`.

Implementation:

- `Scripts/Objects_And_Classes/Opponent_Object_Script.gd`
- `Scripts/Objects_And_Classes/NPC_Object_Script.gd`
- `Scripts/Objects_And_Classes/Shopkeeper_Script.gd`

3.1 Pattern values

Pattern	Behaviour	Extra fields
<code>idle_random</code>	Stands still, randomly turns to face a new direction every 1–4 s	—
<code>idle_cycle</code>	Stands still but cycles the <code>walk_down</code> animation (used for swimmers)	—
<code>idle_up</code> / <code>idle_down</code> / <code>idle_left</code> / <code>idle_right</code>	Stationary facing the given direction; restored 1 s after a conversation	—
<code>patrol_line</code>	Walks back and forth along one axis	<code>patrol_speed</code> (default 60), <code>patrol_distance</code> (default 100), <code>patrol_axis</code> ("horizontal" or "vertical")
<code>patrol_square</code>	Walks a square loop (down → right → up → left → repeat)	<code>patrol_speed</code> , <code>patrol_distance</code>
<code>random_wander</code>	Picks short random walks within a radius of the spawn point	<code>patrol_speed</code> , <code>wander_radius</code> (default 200)

3.2 Extra-field reference

Field	Type	Default	Used by
<code>patrol_speed</code>	float	<code>60.0</code>	<code>patrol_line</code> , <code>patrol_square</code> , <code>random_wander</code>
<code>patrol_distance</code>	float	<code>100.0</code>	<code>patrol_line</code> , <code>patrol_square</code>
<code>patrol_axis</code>	"horizontal" "vertical"	"horizontal"	<code>patrol_line</code>
<code>wander_radius</code>	float	<code>200.0</code>	<code>random_wander</code>

3.3 Examples

```
{ "name": "Biker2", "pattern": "patrol_line", "patrol_speed": 120,
  "patrol_distance": 210, "patrol_axis": "horizontal",
  "position": { "x": 400, "y": 1050 } }

{ "name": "Pikachu Fan Marina", "pattern": "random_wander",
  "patrol_speed": 20, "wander_radius": 65,
  "position": { "x": 1425, "y": 1000 } }

{ "name": "Fisherman Dave", "pattern": "idle_down",
  "position": { "x": -600, "y": 2730 } }
```

4. NPC Types

`npc_type` decides which behaviour script handles the NPC and which scene is instantiated. Set on the NPC's entry in `All_NPC_Constant_Data.json` (or on the per-map placement if it varies by location).

Dispatcher: `MapManager._load_and_spawn_npcs` (≈ line 280) and `MapManager._on_player_npc_interact` (≈ line 553).

<code>npc_type</code>	Behaviour	Required extra fields
(omitted) / <code>"text_only"</code>	Plain NPC — shows <code>meet_text</code> then <code>repeat_text</code> . Optionally a gift NPC if <code>gift_type</code> is set.	—
<code>"juice_vendor"</code>	Opens the juice-bar coin lottery	— (handled inline in <code>MapManager</code>)
<code>"shop"</code>	Opens the corresponding shop scene; uses Shopkeeper state machine	<code>shop_id</code>

4.1 Shop ids

<code>shop_id</code>	Scene loaded	Notes
<code>"card_mart"</code>	<code>Scenes/Main_Menu_Scenes/Pack_Purchase.tscn</code>	Standard pack shop; uses Day-1 starter-set state machine
<code>"rocket_mart"</code>	<code>Scenes/Main_Menu_Scenes/Pack_Purchase.tscn</code>	Forces <code>unlocked_packs = ["base5"]</code>
<code>"coin_mart"</code>	<code>Scenes/Main_Menu_Scenes/Coin_Shop.tscn</code>	Reads <code>coin_shop_inventory.json</code>
<code>"holo_mart"</code>	<code>Scenes/Main_Menu_Scenes/Holo_Rare_Shop.tscn</code>	—

4.2 Example shop NPC

```
"Holo Rare Shopkeeper": {  
  "sprite": "NPC_Holo_Rare_Shopkeeper",  
  "npc_type": "shop",  
  "shop_id": "holo_mart"  
}
```

5. Gifts (NPC-given)

A non-shop NPC becomes a **gift NPC** as soon as `gift_type` is non-empty. The gift is awarded the first time the player talks to them, then the NPC falls back to its `repeat_text` from then on.

Handler: `MapManager._give_gift` (line 619) and `MapManager._prepare_gift_display` (line 666). **Detection:** `NPC_Object_Script.is_gift_npc()`.

5.1 Gift types

gift_type	gift_value format	What it does	Visual reveal
"card"	Comma-separated card ids, e.g. "base1-1, base2-5"	Adds cards to the player's collection via <code>GameState.give_cards</code>	Card-flip animation
"cash"	Decimal string, e.g. "250"	<code>GameState.add_cash(int(value))</code>	Silent
"coin"	Coin filename without <code>.png</code> , e.g. "Pikachu Gold"	<code>GameState.add_coin_to_collection(value)</code>	Coin-flip animation
"costume"	Costume key (case variants tolerated), e.g. "Sailor", "LASS2"	<code>GameState.add_costume_to_collection(value)</code>	Fade-in costume image
"energy_style"	Style name, e.g. "Base1"	Appends to <code>progress["energy_styles"]</code>	Silent
"available_pack"	Single pack id, e.g. "base4"	Appends to <code>progress["packs_unlocked"]</code> — unlocks it for purchase in Card Mart	Silent
"pack"	Comma-separated pack art codes, e.g. "base5_a, base1_b"	Queues <code>PackOpeningManager.open_packs(...)</code> on the next OK press — opens packs immediately	Pack opening sequence
"pack_of_cards"	(Pack name)	Not implemented — currently emits a <code>push_warning</code>	n/a

5.2 Special placeholders seen in data

Placeholder	Meaning
"REPLACECOIN" in <code>gift_value</code>	Used in early/draft data to mark "fill in a real coin later". Treat as a TODO marker.
"REPLACEMUSIC" in <code>music</code> field	Same idea for opponent BGM.

5.3 Examples

```
// Card gift (single)
"Kid Opening Packs": { "sprite": "NPC 02", "gift_type": "card", "gift_value": "base1-20" }

// Multi-card gift
"Card Elder Hamish Day 4": { "sprite": "Elder", "gift_type": "card",
  "gift_value": "base1-1, base1-2, base1-3" }

// Cash
"Man Staring Out To Sea": { "sprite": "Youngcouple2_2",
  "gift_type": "cash", "gift_value": "250" }

// Costume
"Sailor Working On Dock": { "sprite": "Sailor",
  "gift_type": "costume", "gift_value": "Sailor" }

// Unlock a pack for sale in the Card Mart
"Card Mart Rep": { "sprite": "NPC 09",
  "gift_type": "available_pack", "gift_value": "base4" }

// Immediately open one or more packs
"Rocket Pack Gift": { "sprite": "Teamrocket_M",
  "gift_type": "pack", "gift_value": "base5_a, base5_b" }
```

6. Opponent Rewards

Rewards are read after a winning match by `Match_End_Outro_Script.gd` (around lines 283–327). They live on the opponent's entry in `All_NPC_Constant_Data.json` (or on the per-map placement to override).

Field	Format	Awarded	Notes
<code>cash_reward</code>	Integer string, e.g. <code>"150"</code>	Every win	First win is tripled (×3) and the outro label shows <code>(First Win ×3!)</code>
<code>coin_reward</code>	Single coin filename without <code>.png</code> , e.g. <code>"Pikachu Gold"</code>	First win only , and only if the player doesn't already own that coin	Triggers coin-flip animation
<code>card_reward</code>	Comma-separated card ids, e.g. <code>"basep-3, basep-14"</code>	First win only	Added via <code>GameState.give_cards</code>
<code>costume_reward</code>	Comma-separated costume keys, e.g. <code>"Scientist_M, Scientist_F"</code>	First win only	Per key, only added if not already owned
<code>pack_reward</code>	Single pack art code, e.g. <code>"base5_c"</code>	Not currently consumed — present on <code>Rocket Exec Ariana</code> but no script reads it. Treat as planned/not-wired.	
<code>prize_cards</code>	int (1–6)	—	Number of prize cards in the match itself (not a reward)
<code>music</code>	string (set id / <code>"REPLACEMUSIC"</code>)	—	BGM track played during the battle
<code>deck</code>	Deck name (string)	—	Looks up <code>NPC_and_Opponent_Data/Opponent_Deck_Data/<deck>.json</code>

Are packs allowed as a prize? Not via `pack_reward` (currently unused). If you want a battle to award a pack, the workable mechanism is to award a **card** via `card_reward` and trigger pack-opening via a gift NPC elsewhere with `gift_type: "pack"`. The `pack_reward` field is reserved for future wiring.

6.1 Example opponent

```
"Card Expert Nathan": {
  "sprite":      "Cooltrainer_M3",
  "deck":        "Energy Burn",
  "music":       "REPLACEMUSIC",
  "prize_cards": 6,
  "cash_reward": "150",
  "coin_reward": "Charizard Red",
  "card_reward": "basep-3, basep-14",
  "costume_reward": "Cooltrainer_M3"
}
```

7. Map Files: Day & Time Variants

Each overworld map loads a different JSON file depending on the current **date** and **time of day**.

Loader: `Scripts/Map_Scripts/Celeste_Harbour.gd` (≈ line 16) **State source:** `GameState.get_date()` and `GameState.get_time()`.

7.1 Filename convention

`NPC_and_Opponent_Data/<Map>_<Date>_<Time>.json`

Part	Allowed values
<code><Map></code>	<code>Celeste_Harbour</code> , <code>Verdant_Forest</code> , ... (one per overworld scene)
<code><Date></code>	Integer day number — <code>1</code> , <code>2</code> , <code>3</code> , <code>4</code> , <code>5</code> , ...
<code><Time></code>	<code>Day</code> , <code>Evening</code> , <code>Night</code>

So:

```
NPC_and_Opponent_Data/Celeste_Harbour_1_Day.json
NPC_and_Opponent_Data/Celeste_Harbour_3_Night.json
NPC_and_Opponent_Data/Verdant_Forest_4_Evening.json
```

7.2 File top-level shape

```
{
  "npcs":      [ { ...npc placement... } ],
  "opponents": [ { ...opponent placement... } ]
}
```

Both arrays may carry per-entry `condition` blocks (see §2) to toggle entries on or off based on progress.

7.3 Time cycle

Time advances **Day** → **Evening** → **Night** → **Day** (Day rollover bumps `date`). Day-driven map events (e.g. boats appearing, beach blocked, SS Anne docking) are scripted inside the map's `.gd` file rather than the data JSON.

8. Card Sets, Decks & Player Data

8.1 Card data JSON

Each set lives in `Card_Set_Data/<set_id>.json` and is a flat array of card objects.

```
{
  "id": "base1-1",
  "name": "Alakazam",
  "supertype": "Pokémon", // "Pokémon" | "Trainer" | "Energy"
  "subtypes": ["Stage 2"], // "Basic", "Stage 1", "Stage 2", "Baby",
                          // "Special" (energies), "ex"
  "hp": "80", // string, only on Pokémon
  "types": ["Psychic"], // energy type(s) for Pokémon
  "evolvesFrom": "Kadabra",
  "attacks": [ ... ],
  "weaknesses": [ { "type": "Psychic", "value": "x2" } ],
  "retreatCost": ["Colorless", "Colorless", "Colorless"],
  "convertedRetreatCost": 3,
  "rarity": "Rare Holo"
}
```

Card image path: `res://Image_Assets/Card_Image_Library/<set_id>/Large/<card_id>.png`

8.2 Player decks

Player deck files live in `user://Player_Decks/<name>.json` (and starter seeds in `res://Player_Data/Player_Decks/`). The format is a simple array:

```
[
  { "count": 15, "id": "base1-101" },
  { "count": 4, "id": "base1-43" },
  { "count": 2, "id": "base1-27" }
]
```

Field	Type	Notes
<code>id</code>	String	Lowercase <code>set-number</code> id. Must exist in the matching <code>Card_Set_Data/<set>.json</code> .
<code>count</code>	int	Copies of the card in the deck. No engine-enforced cap; deck-builder UI enforces TCG standard limits.

The player's **currently-selected deck name** is the `deck` field of `user://Player_Current_Data.json`.

8.3 Player owned cards

`Player_Data/Player_Owned_Cards/<set>_player_owned_cards.json` tracks how many copies of each card the player owns:

```
{
  "owned_cards": [
    { "card_id": "base1-1", "owned": 0 },
    { "card_id": "base1-2", "owned": 5 }
  ]
}
```

8.4 Opponent decks

Opponent deck files use the same format as player decks and live at

`NPC_and_Opponent_Data/Opponent_Deck_Data/<DeckName>.json`. The opponent's `deck` field (a string like `"Energy Burn"`) is mapped to the filename.

9. Pack Pricing

File: `Card_Set_Data/pack_prices.json` **Reader:** `Scripts/Menu_Scripts/Pack_Purchase_Script.gd`

```
[
  { "pack": "base1", "cost": "100" },
  { "pack": "base5", "cost": "200" },
  { "pack": "neo4", "cost": "550" }
]
```

Field	Notes
<code>pack</code>	Set id (matches <code>Card_Set_Data/<set>.json</code>)
<code>cost</code>	Cash cost, stored as a string but parsed to int

Pack art codes (for gifts and rewards) follow `<set_id>_<letter>`, e.g. `base1_a`, `base1_b`, `base5_c`. Their images live at `res://Image_Assets/Packs/<art_code>.png`.

10. Coin Shop Inventory

File: `NPC_and_Opponent_Data/coin_shop_inventory.json` **Reader:** `Scripts/Menu_Scripts/Coin_Shop_Script.gd`

Currency: Munchlax coins (Silver / Gold / Blue) won at the juice vendor.

```
{
  "coins": [
    { "filename": "Arceus Silver.png", "cost": 300 },
    { "filename": "Gardevoir Silver 2.png", "cost": 300 }
  ]
}
```

Field	Notes
<code>filename</code>	Coin image filename including <code>.png</code> (lives in <code>Image_Assets/Coins/</code>)
<code>cost</code>	Coin cost — integer, not string

11. Save / Progress Data Shape

Persisted at `user://Player_Game_Progress.json` and accessed only via the `GameState` autoload (never bypass it — see `[[project_gamestate_save_invariant]]`).

Key fields you'll see referenced in JSON conditions and gift hooks:

Progress key	Type	Set by
<code>date</code>	int	Time cycle / sleep flow
<code>time</code>	"Day" "Evening" "Night"	Time cycle
<code>cash</code>	int	<code>add_cash</code> , battle rewards
<code>coins</code>	String[] of filenames	<code>add_coin_to_collection</code>
<code>costumes</code>	String[] of lowercase .png keys	<code>add_costume_to_collection</code>
<code>energy_styles</code>	String[]	Gift <code>energy_style</code>
<code>packs_unlocked</code>	String[] of set ids	Gift <code>available_pack</code>
<code>packs_opened_total</code>	int	<code>Pack_Opening_Manager.gd</code> — gates god-pack unlock (>20) and guarantee (every 100). See <code>[[project_pack_opening_counter]]</code> .
<code>opponents_beaten</code>	String[]	Match outro
<code>npcs_met</code>	String[]	First interaction
<code>gifts_received</code>	String[] of NPC names	<code>mark_gift_received</code>
<code>shop_state</code>	"initial" "awaiting_funds" "restocking" "open"	Shopkeeper state machine
<code>player_collected_shop_starter_set</code>	bool	Day-1 starter purchase
<code>shop_free_packs_given</code>	bool	Day-2 free pack drop

Custom `flag_set` / `flag_not_set` conditions read whatever key you point at in this dict, so feel free to invent new boolean flags as long as something elsewhere is setting them.

12. Quick-Reference Cheat Sheet

I want to...	Edit this
Make an opponent require a specific deck	Add <code>restrictions: { ... }</code> to their entry (§1)
Hide an opponent until a prerequisite is beaten	Add <code>condition: { type: "opponent_defeated", target: "..." }</code> (§2)
Change how an NPC moves around	Set <code>pattern</code> plus the matching extra fields (§3)
Add a new shop NPC	<code>npc_type: "shop"</code> plus the correct <code>shop_id</code> (§4)
Give the player a card / costume / cash	<code>gift_type</code> + <code>gift_value</code> on the NPC (§5)
Reward an opponent battle with cash + cards	<code>cash_reward</code> + <code>card_reward</code> (§6)
Add a new map-day/time variant	Create <code><Map>_<Day>_<Time>.json</code> (§7)
Add a new card to a set	Append to <code>Card_Set_Data/<set>.json</code> (§8.1)
Change a pack's price	Edit <code>Card_Set_Data/pack_prices.json</code> (§9)
Add a coin to the Coin Shop	Append to <code>coin_shop_inventory.json</code> (§10)

